MYSQL REVISION TOUR

MYSQL

It is freely available open source Relational Database Management System (RDBMS) that uses **Structured Query Language(SQL).** In MySQL database , information is stored in Tables. A single MySQL database can contain many tables at once and store thousands of individual records.

SQL (Structured Query Language)

SQL is a language that enables you to create and operate on relational databases, which are sets of related information stored in tables.

DIFFERENT DATA MODELS

A **data model refers to a set of concepts to describe the structure of a database**, and certain constraints (restrictions) that the database should obey. The four data model that are used for database management are :

1. **Relational data model** : In this data model, the data is organized into tables (i.e. rows and columns). These tables are called relations.

2. **Hierarchical data model 3. Network data model 4. Object Oriented data model**

RELATIONAL MODEL TERMINOLOGY

1. **Relation :** A table storing logically related data is called a Relation.

2. **Tuple :** A **row of a relation** is generally referred to as a tuple.

3. **Attribute :** A **column** of a relation is generally referred to as an attribute/Field.

4. **Degree :** This refers to the **number of attributes** in a relation.

5. **Cardinality :** This refers to the **number of tuples** in a relation.

6. **Primary Key :** This refers to a set of one or more attributes that can uniquely identify tuples within the relation.

7. **Candidate Key :** All attribute combinations inside a relation that can serve as primary key are candidate keys(as these are candidates for primary key position).

8. **Alternate Key :** A candidate key that is not primary key, is called an alternate key.

9. **Foreign Key :** A non-key attribute, whose values are derived from the primary key of some other table, is known as foreign key in its current table.

REFERENTIAL INTEGRITY

- A referential integrity is a system of rules that a DBMS uses to ensure that relationships between records in related tables are valid, and that users don't accidentally delete or change related data. This integrity is ensured by foreign key.

CLASSIFICATION OF SQL STATEMENTS

SQL commands can be mainly divided into following categories:

1. Data Definition Language(DDL) Commands

Commands that allow you to perform task, related to data definition e.g;

- Creating, altering and dropping.
- Granting and revoking privileges and roles.
- Maintenance commands.

2. **Data Manipulation Language(DML) Commands**

   Commands that allow you to perform data manipulation e.g., retrieval, insertion, deletion and modification of data stored in a database.

3. **Transaction Control Language(TCL) Commands**

   Commands that allow you to manage and control the transactions e.g.,

   - Making changes to database, permanent
   - Undoing changes to database, permanent
   - Creating savepoints
   - Setting properties for current transactions.

MySQL ELEMENTS

1. Literals 2. Datatypes 3. Nulls 4. Comments

LITERALS

It refer to a fixed data value. This fixed data value may be of character type or numeric type. For example, 'replay' , 'Raj', '8' , '306' are all character literals.

Numbers not enclosed in quotation marks are numeric literals. E.g. 22 , 18 , 1997 are all numeric literals.

Numeric literals can either be integer literals i.e., without any decimal or be real literals i.e. with a decimal point e.g. 17 is an integer literal but 17.0 and 17.5 are real literals.

DATA TYPES

Data types are means to identify the type of data and associated operations for handling it. MySQL data types are divided into three categories:

- Numeric
- Date and time
- String types

Numeric Data Type

1. int – used for number without decimal.
2. Decimal(m,d) – used for floating/real numbers. m denotes the total length of number and d is number of decimal digits.

Date and Time Data Type

1. date – used to store date in YYYY-MM-DD format.
2. time – used to store time in HH:MM:SS format.

String Data Types

1. char(m) – used to store a fixed length string. **m** denotes max. number of characters.
2. varchar(m) – used to store a variable length string. **m** denotes max. no. of characters.

DIFFERENCE BETWEEN CHAR AND VARCHAR DATA TYPE

S.NO.	Char Datatype	Varchar Datatype
1.	It specifies a **fixed length** character String.	It specifies a **variable length** character string.
2.	When a column is given datatype as CHAR(**n**), then MySQL ensures that all values stored in that column have this length i.e. **n** bytes. If a value is shorter than this length **n** then blanks are added, but the size of value remains **n** bytes.	When a column is given datatype as VARCHAR(**n**), then the maximum size a value in this column can have is **n** bytes. Each value that is stored in this column store exactly as you specify it i.e. no blanks are added if the length is shorter than maximum length **n**.

NULL VALUE

If a column in a row has no value, then column is said to be **null** , or to contain a null. **You should use a null value** when the actual value is not known or when a value would not be meaningful.

DATABASE COMMNADS

1. VIEW EXISTING DATABASE

To view existing database names, the command is : **SHOW DATABASES ;**

2. CREATING DATABASE IN MYSQL

For creating the database in MySQL, we write the following command : **CREATE DATABASE** <databasename> ;

e.g. In order to create a database Student, command is :

CREATE DATABASE Student ;

3. ACCESSING DATABASE

For accessing already existing database , we write :

USE <databasename> ;

e.g. to access a database named Student , we write command as :

USE Student ;

4. DELETING DATABASE

For deleting any existing database , the command is :

DROP DATABASE <databasename> ;

e.g. to delete a database , say student, we write command as :

DROP DATABASE Student ;

5. VIEWING TABLE IN DATABASE

In order to view tables present in currently accessed database , command is : **SHOW TABLES ;**

CREATING TABLES IN MYSQL

- Tables are created with the CREATE TABLE command. When a table is created, its columns are named, data types and sizes are supplied for each column.

Syntax of CREATE TABLE command is : CREATE TABLE <table-name>

(<column name> <data type> ,
<column name> <data type> ,
…… **) ;**

E.g. in order to create table EMPLOYEE given below :

ECODE	ENAME	GENDER	GRADE	GROSS

We write the following command :

```
CREATE TABLE employee
(    ECODE integer ,
     ENAME varchar(20) ,
     GENDER char(1) ,
     GRADE char(2) ,
     GROSS integer    ) ;
```

INSERTING DATA INTO TABLE

- The rows are added to relations(table) using INSERT command of SQL. Syntax of
  INSERT is : INSERT INTO <tablename> [<column list>]
  VALUE (<value1> , <value2> , …..) ;

e.g. to enter a row into EMPLOYEE table (created above), we write command as :

```
INSERT INTO employee
VALUES(1001 , 'Ravi' , 'M' , 'E4' , 50000);
```

OR

```
INSERT INTO employee (ECODE , ENAME , GENDER , GRADE , GROSS)
VALUES(1001 , 'Ravi' , 'M' , 'E4' , 50000);
```

ECODE	ENAME	GENDER	GRADE	GROSS
1001	Ravi	M	E4	50000

In order to insert another row in EMPLOYEE table , we write again INSERT command :

```
INSERT INTO employee
VALUES(1002 , 'Akash' , 'M' , 'A1' , 35000);
```

ECODE	ENAME	GENDER	GRADE	GROSS
1001	Ravi	M	E4	50000
1002	Akash	M	A1	35000

INSERTING NULL VALUES

- To insert value NULL in a specific column, we can type NULL without quotes and NULL will be inserted in that column. E.g. in order to insert NULL value in ENAME column of above table, we write INSERT command as :

```
INSERT   INTO EMPLOYEE
VALUES (1004 , NULL , 'M' , 'B2' , 38965 ) ;
```

ECODE	ENAME	GENDER	GRADE	GROSS
1001	Ravi	M	E4	50000
1002	Akash	M	A1	35000
1004	NULL	M	B2	38965

SIMPLE QUERY USING SELECT COMMAND

- The SELECT command is used to pull information from a table. Syntax of SELECT command is : SELECT <column name>,<column name>
  FROM <tablename>
  WHERE <condition name> ;

SELECTING ALL DATA

- In order to retrieve everything (all columns) from a table, SELECT command is used as : **SELECT * FROM** <tablename> ;

e.g. In order to retrieve everything from **Employee** table, we write SELECT command as :

SELECT * FROM Employee ;

EMPLOYEE

ECODE	ENAME	GENDER	GRADE	GROSS
1001	Ravi	M	E4	50000
1002	Akash	M	A1	35000
1004	NULL	M	B2	38965

SELECTING PARTICULAR COLUMNS

EMPLOYEE

ECODE	ENAME	GENDER	GRADE	GROSS
1001	Ravi	M	E4	50000
1002	Akash	M	A1	35000
1004	Neela	F	B2	38965
1005	Sunny	M	A2	30000
1006	Ruby	F	A1	45000
1009	Neema	F	A2	52000

- A particular column from a table can be selected by specifying column-names with SELECT command. E.g. in above table, if we want to select ECODE and ENAME column, then command is :

SELECT ECODE , ENAME
FROM EMPLOYEE ;

E.g.2 in order to select only ENAME, GRADE and GROSS column, the command is :

SELECT ENAME , GRADE , GROSS
FROM EMPLOYEE ;

SELECTING PARTICULAR ROWS

We can select particular rows from a table by specifying a condition through **WHERE clause** along with SELECT statement. **E.g.** In employee table if we want to select rows where Gender is female, then command is :

SELECT * FROM **EMPLOYEE**
WHERE **GENDER = 'F' ;**

E.g.2. in order to select rows where salary is greater than 48000, then command is :

SELECT * FROM **EMPLOYEE**
WHERE **GROSS > 48000 ;**

ELIMINATING REDUNDANT DATA

The **DISTINCT** keyword eliminates duplicate rows from the results of a SELECT statement. For example ,

SELECT GENDER **FROM** EMPLOYEE ;

GENDER
M
M
F
M
F
F

SELECT DISTINCT(GENDER) **FROM** EMPLOYEE ;

DISTINCT(GENDER)
M
F

VIEWING STRUCTURE OF A TABLE

- If we want to know the structure of a table, we can use DESCRIBE or DESC command, as per following syntax :

DESCRIBE | **DESC** <tablename> ;

e.g. to view the structure of table **EMPLOYEE**, command is : **DESCRIBE** EMPLOYEE ; **OR DESC** EMPLOYEE ;

USING COLUMN ALIASES

- The columns that we select in a query can be given a different name, i.e. column alias name for output purpose.

Syntax :

SELECT <columnname> **AS** column alias , <columnname> **AS** column alias …..
FROM <tablename> ;

e.g. In output, suppose we want to display ECODE column as EMPLOYEE_CODE in output , then command is :

SELECT **ECODE AS "EMPLOYEE_CODE"**
FROM **EMPLOYEE ;**

CONDITION BASED ON A RANGE

- The **BETWEEN** operator defines a range of values that the column values must fall in to make the condition true. The range include both lower value and upper value.

e.g. to display ECODE, ENAME and GRADE of those employees whose salary is between 40000 and 50000, command is:

SELECT ECODE , ENAME ,GRADE
FROM **EMPLOYEE**
WHERE GROSS **BETWEEN** 40000 AND50000 ;

Output will be :

ECODE	ENAME	GRADE
1001	Ravi	E4
1006	Ruby	A1

CONDITION BASED ON A LIST

- To specify a list of values, IN operator is used. The IN operator selects value that match any value in a given list of values. E.g.

SELECT * FROM EMPLOYEE
WHERE GRADE **IN** ('A1' , 'A2');

Output will be :

ECODE	**ENAME**	**GENDER**	**GRADE**	**GROSS**
1002	Akash	M	A1	35000
1006	Ruby	F	A1	45000
1005	Sunny	M	A2	30000
1009	Neema	F	A2	52000

- The **NOT IN** operator finds rows that do not match in the list. E.g.

SELECT * FROM EMPLOYEE
WHERE GRADE **NOT IN** ('A1' , 'A2');

Output will be :

ECODE	**ENAME**	**GENDER**	**GRADE**	**GROSS**
1001	Ravi	M	E4	50000
1004	Neela	F	B2	38965

CONDITION BASED ON PATTERN MATCHES

- LIKE operator is used for pattern matching in SQL. Patterns are described using two special wildcard characters:
  1. percent(%) – The % character matches any substring.
  2. underscore(_) – The _ character matches any character.

e.g. to display names of employee whose name starts with R in EMPLOYEE table, the command is :

```
SELECT ENAME
FROM EMPLOYEE
WHERE ENAME LIKE 'R%' ;
```

Output will be :

ENAME
Ravi
Ruby

e.g. to display details of employee whose second character in name is 'e'.

```
SELECT *
FROM EMPLOYEE
WHERE ENAME LIKE '_e%' ;
```

Output will be :

ECODE	ENAME	GENDER	GRADE	GROSS
1004	Neela	F	B2	38965
1009	Neema	F	A2	52000

e.g. to display details of employee whose name ends with 'y'.

```
SELECT *
FROM EMPLOYEE
WHERE ENAME LIKE '%y' ;
```

Output will be :

ECODE	ENAME	GENDER	GRADE	GROSS
1005	Sunny	M	A2	30000
1006	Ruby	F	A1	45000

SEARCHING FOR NULL

- The NULL value in a column can be searched for in a table using IS NULL in the WHERE clause. E.g. to list employee details whose salary contain NULL, we use the command :

```
SELECT *
FROM EMPLOYEE WHERE
GROSS IS NULL ;
```

e.g.

STUDENT

Roll_No	Name	Marks
1	ARUN	NULL
2	RAVI	56
4	SANJAY	NULL

to display the names of those students whose marks is NULL, we use the command :

```
SELECT Name
FROM EMPLOYEE
WHERE Marks IS NULL ;
```

Output will be :

Name
ARUN
SANJAY

SORTING RESULTS

Whenever the SELECT query is executed , the resulting rows appear in a predecided order.The **ORDER BY clause** allow sorting of query result. The sorting can be done either in ascending or descending order, the default is ascending.

The **ORDER BY clause is used as :**

SELECT <column name> , <column name>….
FROM <tablename>
WHERE <condition>
ORDER BY <column name> ;

e.g. to display the details of employees in EMPLOYEE table in alphabetical order, we use command :

SELECT *
FROM EMPLOYEE
ORDER BY ENAME ;

Output will be :

ECODE	ENAME	GENDER	GRADE	GROSS
1002	Akash	M	A1	35000
1004	Neela	F	B2	38965
1009	Neema	F	A2	52000
1001	Ravi	M	E4	50000
1006	Ruby	F	A1	45000
1005	Sunny	M	A2	30000

e.g. display list of employee in descending alphabetical order whose salary is greater than 40000.

SELECT ENAME
FROM EMPLOYEE
WHERE GROSS > 40000
ORDER BY ENAME desc ;

Output will be :

ENAME
Ravi
Ruby
Neema

MODIFYING DATA IN TABLES

you can modify data in tables using UPDATE command of SQL. The UPDATE command specifies the rows to be changed using the WHERE clause, and the new data using the SET keyword. Syntax of update command is :

UPDATE <tablename>
SET <columnname>=value , <columnname>=value
WHERE <condition> ;

e.g. to change the salary of employee of those in EMPLOYEE table having employee code 1009 to 55000.

UPDATE EMPLOYEE
SET GROSS = 55000
WHERE ECODE = 1009 ;

UPDATING MORE THAN ONE COLUMNS

e.g. to update the salary to 58000 and grade to B2 for those employee whose employee code is 1001.

UPDATE EMPLOYEE
SET GROSS = 58000, GRADE='B2'
WHERE ECODE = 1009 ;

OTHER EXAMPLES

e.g.1. Increase the salary of each employee by 1000 in the EMPLOYEE table.

UPDATE EMPLOYEE
SET GROSS = GROSS +100 ;

e.g.2. Double the salary of employees having grade as 'A1' or 'A2' .

UPDATE EMPLOYEE
SET GROSS = GROSS * 2 ;
WHERE GRADE='A1' **OR** GRADE='A2' ;

e.g.3. Change the grade to 'A2' for those employees whose employee code is 1004 and name is Neela.

UPDATE EMPLOYEE
SET GRADE='A2'
WHERE ECODE=1004 **AND** GRADE='NEELA' ;

DELETING DATA FROM TABLES

To delete some data from tables, DELETE command is used. **The DELETE command removes rows from a table.** The syntax of DELETE command is :

DELETE FROM <tablename>
WHERE <condition> ;

For example, to remove the details of those employee from EMPLOYEE table whose grade is A1.

DELETE FROM EMPLOYEE
WHERE GRADE ='A1' ;

TO DELETE ALL THE CONTENTS FROM A TABLE

DELETE FROM EMPLOYEE ;

So if we do not specify any condition with WHERE clause, then all the rows of the table will be deleted. Thus above line will delete all rows from employee table.

DROPPING TABLES

The DROP TABLE command lets you drop a table from the database. The **syntax of DROP TABLE** command is :

DROP TABLE <tablename> ;

e.g. to drop a table employee, we need to write :

DROP TABLE employee ;

Once this command is given, the table name is no longer recognized and no more commands can be given on that table. After this command is executed, all the data in the table along with table structure will be deleted.

S.NO.	DELETE COMMAND	DROP TABLE COMMAND
1	It is a DML command.	It is a DDL Command.
2	This command is used to delete only rows of data from a table	This command is used to delete all the data of the table along with the structure of the table. The table is no longer recognized when this command gets executed.
3	Syntax of DELETE command is: **DELETE FROM** <tablename> **WHERE** <condition> ;	Syntax of DROP command is : **DROP TABLE** <tablename> ;

ALTER TABLE COMMAND

The ALTER TABLE command is used to change definitions of existing tables.(adding columns,deleting columns etc.). The ALTER TABLE command is used for :

1. Adding columns to a table
2. Modifying column-definitions of a table.
3. Deleting columns of a table.
4. Adding constraints to table.
5. Enabling/Disabling constraints.

ADDING COLUMNS TO TABLE

To add a column to a table, ALTER TABLE command can be used as per following syntax:

ALTER TABLE <tablename>
ADD <Column name> <datatype> <constraint> ;

e.g. to add a new column ADDRESS to the EMPLOYEE table, we can write command as :

ALTER TABLE EMPLOYEE
ADD ADDRESS VARCHAR(50);

A new column by the name ADDRESS will be added to the table, where each row will contain NULL value for the new column.

ECODE	ENAME	GENDER	GRADE	GROSS	ADDRESS
1001	Ravi	M	E4	50000	NULL
1002	Akash	M	A1	35000	NULL
1004	Neela	F	B2	38965	NULL
1005	Sunny	M	A2	30000	NULL
1006	Ruby	F	A1	45000	NULL
1009	Neema	F	A2	52000	NULL

However **if you specify NOT NULL constraint while adding a new column**, MySQL adds the new column with the default value of that datatype e.g. for INT type it will add 0 , for CHAR types, it will add a space, and so on.
e.g. Given a table namely Testt with the following data in it.

Col1	Col2
1	A
2	G

Now following commands are given for the table. Predict the table contents after each of the following statements:

(i) ALTER TABLE testt ADD col3 INT ;
(ii) ALTER TABLE testt ADD col4 INT NOT NULL ;
(iii) ALTER TABLE testt ADD col5 CHAR(3) NOT NULL ;
(iv) ALTER TABLE testt ADD col6 VARCHAR(3);

MODIFYING COLUMNS

Column name and data type of column can be changed as per following syntax :

ALTER TABLE <table name>
CHANGE <old column name> <new column name> <new datatype>;

If **Only data type of column need to be changed**, then

ALTER TABLE <table name>
MODIFY <column name> <new datatype>;

e.g.1. In table EMPLOYEE, change the column GROSS to SALARY.

ALTER TABLE EMPLOYEE
CHANGEGROSS SALARYINTEGER;

e.g.2. In table EMPLOYEE , change the column ENAME to EM_NAME and data type from VARCHAR(20) to VARCHAR(30).

ALTER TABLE EMPLOYEE
CHANGE ENAME EM_NAME VARCHAR(30);

e.g.3. In table EMPLOYEE , change the datatype of GRADE column from CHAR(2) to VARCHAR(2).

ALTER TABLE EMPLOYEE
MODIFY GRADE VARCHAR(2);

DELETING COLUMNS

To delete a column from a table, the ALTER TABLE command takes the following form :

ALTER TABLE <table name>
DROP <column name>;

e.g. to delete column GRADE from table EMPLOYEE, we will write :

ALTER TABLE EMPLOYEE
DROP GRADE ;

ADDING/REMOVING CONSTRAINTS TO A TABLE

ALTER TABLE statement can be used to add constraints to your existing table by using it in following manner:

- **TO ADD PRIMARY KEY CONSTRAINT**

  ALTER TABLE <table name>
  ADD PRIMARY KEY (Column name);

e.g. to add PRIMARY KEY constraint on column ECODE of table EMPLOYEE , the command is :

ALTER TABLE EMPLOYEE
ADD PRIMARY KEY (ECODE) ;

- **TO ADD FOREIGN KEY CONSTRAINT**

  ALTER TABLE <table name>
  ADD FOREIGN KEY (Column name) REFERENCES Parent Table (Primary key of Parent Table);

REMOVING CONSTRAINTS

- To remove primary key constraint from a table, we use ALTER TABLE command as :

  ALTER TABLE <table name> **DROP** PRIMARY KEY ;

- To remove foreign key constraint from a table, we use ALTER TABLE command as :

  ALTER TABLE <table name> **DROP** FOREIGN KEY ;

ENABLING/DISABLING CONSTRAINTS

Only foreign key can be disabled/enabled in MySQL.

To disable foreign keys : **SET** FOREIGN_KEY_CHECKS = 0 ;
To enable foreign keys : **SET** FOREIGN_KEY_CHECKS = 1 ;

INTEGRITY CONSTRAINTS/CONSTRAINTS

- A constraint is a condition or check applicable on a field(column) or set of fields(columns).
- Common types of constraints include :

S.No.	Constraints	Description
1	NOT NULL	Ensures that a column cannot have NULL value
2	DEFAULT	Provides a default value for a column when none is specified
3	UNIQUE	Ensures that all values in a column are different
4	CHECK	Makes sure that all values in a column satisfy certain criteria
5	PRIMARY KEY	Used to uniquely identify a row in the table
6	FOREIGN KEY	Used to ensure referential integrity of the data

NOT NULL CONSTRAINT

By default, a column can hold NULL. It you not want to allow NULL value in a column, then NOT NULL constraint must be applied on that column. E.g.

```
CREATE TABLE Customer
(       SID integer NOT NULL ,
        Last_Name varchar(30)   NOT NULL ,
        First_Name varchar(30)      ) ;
```

Columns **SID** and **Last_Name** cannot include NULL, while **First_Name** can include NULL.

An attempt to execute the following SQL statement,

```
INSERT INTO Customer
VALUES (NULL , 'Kumar' , 'Ajay');
```

will result in an error because this will lead to column SID being NULL, which violates the NOT NULL constraint on that column.

DEFAULT CONSTARINT

The DEFAULT constraint provides a default value to a column when the INSERT INTO statement does not provide a specific value. **E.g.**

```
CREATE TABLE Student
( Student_ID integer ,
   Name varchar(30) ,
   Score integer DEFAULT 80);
```

When following SQL statement is executed on table created above:

```
INSERT INTO Student
VALUES (10 , 'Ravi' );
```

no value has been provided for score field.

Then table **Student** looks like the following:

Student_ID	Name	Score
10	Ravi	**80**

score field has got the default value

UNIQUE CONSTRAINT

- The UNIQUE constraint ensures that all values in a column are distinct. In other words, no two rows can hold the same value for a column with UNIQUE constraint.

e.g.

```
CREATE TABLE Customer
(       SID integer Unique ,
        Last_Name varchar(30) ,
        First_Name varchar(30) ) ;
```

Column SID has a unique constraint, and hence cannot include duplicate values. So, if the table already contains the following rows :

SID	Last_Name	First_Name
1	Kumar	Ravi
2	Sharma	Ajay
3	Devi	Raj

The executing the following SQL statement,

```
INSERT INTO Customer
VALUES ('3' , 'Cyrus' , 'Grace') ;
```

will result in an error because the value 3 already exist in the SID column, thus trying to insert another row with that value violates the UNIQUE constraint.

CHECK CONSTRAINT

- The CHECK constraint ensures that all values in a column satisfy certain conditions. Once defined, the table will only insert a new row or update an existing row if the new value satisfies the CHECK constraint.

e.g.

```
CREATE TABLE Customer
(       SID integer CHECK (SID > 0),
        Last_Name varchar(30) ,
        First_Name varchar(30) ) ;
```

So, attempting to execute the following statement :

```
INSERT INTO Customer
VALUES (-2 , 'Kapoor' , 'Raj');
```

will result in an error because the values for SID must be greater than 0.

PRIMARY KEY CONSTRAINT

- A primary key is used to identify each row in a table. A primary key can consist of one or more fields(column) on a table. When multiple fields are used as a primary key, they are called a **composite key.**

- You can define a primary key in CREATE TABLE command through keywords PRIMARY KEY. e.g.

```
CREATE TABLE Customer
(       SID integer NOT NULL PRIMARY KEY,
        Last_Name varchar(30) ,
        First_Name varchar(30) ) ;
```

Or

```
CREATE TABLE Customer
(       SID integer,
        Last_Name varchar(30) ,
        First_Name varchar(30),
        PRIMARY KEY (SID) ) ;
```

- The latter way is useful if you want to specify a composite primary key, **e.g.**

```
CREATE TABLE Customer
(       Branch integer NOT NULL,
        SID integer NOT NULL ,
        Last_Name varchar(30) ,
        First_Name varchar(30),
        PRIMARY KEY (Branch , SID) ) ;
```

FOREIGN KEY CONSTRAINT

- Foreign key is a non key column of a table (**child table**) that draws its values from **primary key** of another table(**parent table).**
- The table in which a foreign key is defined is called a **referencing table or child table.** A table to which a foreign key points is called **referenced table or parent table.**

e.g.

TABLE: STUDENT ← **Parent Table**

ROLL_NO ← **Primary key**	NAME	CLASS
1	ABC	XI
2	DEF	XII
3	XYZ	XI

Child Table

TABLE: SCORE

ROLL_NO	MARKS
1	55
2	83
3	90

Here column Roll_No is a foreign key in table SCORE(Child Table) and it is drawing its values from Primary key (ROLL_NO) of STUDENT table.(Parent Key).

```
CREATE TABLE STUDENT
(   ROLL_NO integer NOT NULL PRIMARY KEY ,
    NAME VARCHAR(30) ,
    CLASS VARCHAR(3) );

    CREATE TABLE SCORE
    (       ROLL_NO integer ,
            MARKS integer ,
            FOREIGN KEY(ROLL_NO) REFERNCES STUDENT(ROLL_NO) ) ;
```

**** Foreign key is always defined in the child table.***

Syntax for using foreign key

FOREIGN KEY(column name) REFERENCES Parent_Table(PK of Parent Table);

REFERENCING ACTIONS

Referencing action with ON DELETE clause determines what to do in case of a DELETE occurs in the parent table.
Referencing action with ON UPDATE clause determines what to do in case of a UPDATE occurs in the parent table.

Actions:

1. **CASCADE :** This action states that if a DELETE or UPDATE operation affects a row from the parent table, then automatically delete or update the matching rows in the child table i.e., cascade the action to child table.
2. **SET NULL :** This action states that if a DELETE or UPDATE operation affects a row from the parent table, then set the foreign key column in the child table to NULL.
3. **NO ACTION :** Any attempt for DELETE or UPDATE in parent table is not allowed.
4. **RESTRICT :** This action rejects the DELETE or UPDATE operation for the parent table.

Q: Create two tables

Customer(customer_id, name)
Customer_sales(transaction_id, amount , **customer_id**)

Underlined columns indicate primary keys and bold column names indicate foreign key.
Make sure that no action should take place in case of a DELETE or UPDATE in the parent table.

Sol : CREATE TABLE Customer (
customer_id int Not Null Primary Key ,
name varchar(30)) ;

```
CREATE TABLE Customer_sales (
    transaction_id Not Null Primary Key ,
    amount int ,
    customer_id int ,
    FOREIGN KEY(customer_id) REFERENCES Customer (customer_id)
    ON DELETE NO ACTION
    ON UPDATE NO ACTION );
```

Q: Distinguish between a Primary Key and a Unique key in a table.

S.NO.	PRIMARY KEY	UNIQUEKEY
1.	Column having Primary key can't contain NULL value	Column having Unique Key can contain NULL value
2.	There can be only one primary key in Table.	Many columns can be defined as Unique key

Q: Distinguish between ALTER Command and UPDATE command of SQL.

S.NO.	ALTER COMMAND	UPDATE COMMAND
1.	It is a DDL Command	It is a DML command
2.	It is used to change the definition of existing table, i.e. adding column, deleting column, etc.	It is used to modify the data values present in the rows of the table.
3.	Syntax for adding column in a table: ALTER TABLE <tablename> ADD <Column name><Datatype> ;	Syntax for using UPDATE command: UPDATE <Tablename> SET <Columnname>=value WHERE <Condition> ;

AGGREGATE / GROUP FUNCTIONS

Aggregate / Group functions work upon groups of rows , rather than on single row, and return one single output. Different aggregate functions are : COUNT( ) , AVG( ) , MIN( ) , MAX( ) , SUM ( )

Table : EMPL

EMPNO	ENAME	JOB	SAL	DEPTNO
8369	SMITH	CLERK	2985	10
8499	ANYA	SALESMAN	9870	20
8566	AMIR	SALESMAN	8760	30
8698	BINA	MANAGER	5643	20
8912	SUR	NULL	3000	10

1. AVG( )

This function computes the average of given data. e.g. SELECT AVG(SAL)
FROM EMPL ;

Output

AVG(SAL)
6051.6

2. COUNT( )

This function counts the number of rows in a given column.

If you specify the COLUMN name in parenthesis of function, then this function returns rows where COLUMN is not null.
If you specify the asterisk (*), this function returns all rows, including duplicates and nulls.

e.g. SELECT COUNT(*)
FROM EMPL ;

Output

COUNT(*)
5

e.g.2 SELECT COUNT(JOB)
FROM EMPL ;

Output

COUNT(JOB)
4

3. MAX( )

This function returns the maximum value from a given column or expression.

e.g. SELECT MAX(SAL)
FROM EMPL ;

Output

MAX(SAL)
9870

4. MIN( )

This function returns the minimum value from a given column or expression.

e.g. SELECT MIN(SAL)
FROM EMPL ;

Output

MIN(SAL)
2985

5. SUM( )

This function returns the sum of values in given column or expression.

e.g. SELECT SUM(SAL)
FROM EMPL ;

Output

SUM(SAL)
30258

GROUPING RESULT – GROUP BY

The GROUP BY clause combines all those records(row) that have identical values in a particular field(column) or a group of fields(columns).

GROUPING can be done by a column name, or with aggregate functions in which case the aggregate produces a value for each group.

Table : EMPL

EMPNO	ENAME	JOB	SAL	DEPTNO
8369	SMITH	CLERK	2985	10
8499	ANYA	SALESMAN	9870	20
8566	AMIR	SALESMAN	8760	30
8698	BINA	MANAGER	5643	20

e.g. Calculate the number of employees in each grade.

SELECT JOB, COUNT(*)
FROM EMPL
GROUP BY JOB;

Output

JOB	COUNT(*)
CLERK	1
SALESMAN	2
MANAGER	1

e.g.2. Calculate the sum of salary for each department.

SELECT DEPTNO , SUM(SAL)
FROM EMPL
GROUP BY DEPTNO ;

Output

DEPTNO	SUM(SAL)
10	2985
20	15513
30	8760

e.g.3. find the average salary of each department.

Sol:_____

_____

** One thing that you should keep in mind is that while grouping , you should include only those values in the SELECT list that either have the same value for a group or contain a group(aggregate) function. Like in e.g. 2 given above, DEPTNO column has one(same) value for a group and the other expression SUM(SAL) contains a group function.*

NESTED GROUP

- To create a group within a group i.e., nested group, you need to specify multiple fields in the GROUP BY expression. e.g. To group records **job wise** within **Deptno wise**, you need to issue a query statement like :

```
SELECT DEPTNO , JOB, COUNT(EMPNO)
FROM EMPL
GROUP BY DEPTNO , JOB ;
```

Output

DEPTNO	JOB	COUNT(EMPNO)
10	CLERK	1
20	SALESMAN	1
20	MANAGER	1
30	SALESMAN	1

PLACING CONDITION ON GROUPS – HAVING CLAUSE

- The **HAVING clause places conditions on groups** in contrast to WHERE clause that places condition on individual rows. While **WHERE conditions cannot include aggregate functions, HAVING conditions can do so.**
- e.g. To display the jobs where the number of employees is less than 2,

```
SELECT JOB, COUNT(*)
FROM EMPL
GROUP BY JOB
HAVINGCOUNT(*) < 2 ;
```

Output

JOB	COUNT(*)
CLERK	1
MANAGER	1

MySQL FUNCTIONS

A function is a special type of predefined command set that performs some operation and returns a single value. Types of MySQL functions : String Functions , Maths Functions and Date & Time Functions.

Table : EMPL

EMPNO	ENAME	JOB	SAL	DEPTNO
8369	SMITH	CLERK	2985	10
8499	ANYA	SALESMAN	9870	20
8566	AMIR	SALESMAN	8760	30
8698	BINA	MANAGER	5643	20
8912	SUR	NULL	3000	10

STRING FUNCTIONS

1. CONCAT( ) - Returns the Concatenated String.

Syntax : CONCAT(Column1 , Column2 , Column3, …….)

e.g. SELECT CONCAT(EMPNO , ENAME) FROM EMPL WHERE DEPTNO=10;

Output

CONCAT(EMPNO , ENAME)
8369SMITH
8912SUR

2. LOWER( ) / LCASE( ) - Returns the argument in lowercase. Syntax : LOWER(Column name)

e.g.

SELECT LOWER(ENAME) FROM EMPL ;

Output

LOWER(ENAME)
smith
anya
amir
bina
sur

3. UPPER( ) / UCASE( ) - Returns the argument in uppercase. Syntax : UPPER(Column name)

e.g.

SELECT UPPER(ENAME) FROM EMPL ;

Output

UPPER(ENAME)
SMITH
ANYA
AMIR
BINA
SUR

4. SUBSTRING( ) / SUBSTR( ) – Returns the substring as specified.

Syntax : SUBSTR(Column name, m , n), where **m specifies starting index** and **n specifies number of characters from the starting index m.**

e.g.

SELECT SUBSTR(ENAME,2,2) FROM EMPL WHERE DEPTNO=20;

Output

SUBSTR(ENAME,2,2)
NY
IN

SELECT SUBSTR(JOB,-2,2) FROMEMPL WHERE DEPTNO=20;

Output

SUBSTR(JOB,-4,2)
SM
AG

5. LTRIM( ) – Removes leading spaces.

e.g. SELECT LTRIM(' RDBMS MySQL') ;

Output

LTRIM(' RDBMS MySQL')
RDBMS MySQL

6. RTRIM( ) – Removes trailing spaces.

e.g. SELECT RTRIM(‘ RDBMS MySQL ’) ;

Output

RTRIM(‘ RDBMS MySQL’)
RDBMS MySQL

7. TRIM( ) – Removes trailing and leading spaces.

e.g. SELECT TRIM(‘ RDBMS MySQL ’) ;

Output

TRIM(‘ RDBMS MySQL’)
RDBMS MySQL

8. LENGTH( ) – Returns the length of a string. e.g.

SELECT LENGTH(“CANDID”) ;

Output

LENGTH(“CANDID”)
6

e.g.2.

SELECT LENGTH(ENAME) FROM EMPL;

Output

LENGTH(ENAME)
5
4
4
4
3

9. LEFT( ) – Returns the leftmost number of characters as specified.

e.g. SELECT LEFT(‘CORPORATEFLOOR’ , 3) ;

Output

LEFT(‘CORPORATE FLOOR’, 3)
COR

10. RIGHT( ) – Returns the rightmost number of characters as specified.

e.g. SELECT RIGHT(‘CORPORATEFLOOR’ , 3) ;

Output

RIGHT(‘CORPORATE FLOOR’, 3)
OOR

11. MID( ) – This function is same as SUBSTRING( ) / SUBSTR( ) function. E.g.

SELECT MID(“ABCDEF” , 2 , 4) ;

Output

MID(“ABCDEF” , 2 , 4)
BCDE

NUMERIC FUNCTIONS

These functions accept numeric values and after performing the operation, return numeric value.

1. MOD( ) – Returns the remainder of given two numbers. e.g. SELECT MOD(11 , 4) ;**Output**

MOD(11, 4)
3

2. POW( ) / POWER( ) - This function returns m^n i.e , a number m raised to the n^{th} power.

e.g. SELECT POWER(3,2) ;

Output

POWER(3, 2)
9

3. ROUND( ) – This function returns a number rounded off as per given specifications. e.g. ROUND(15.193 , 1) ;

Output

ROUND(15.193 , 1)
15.2

e.g. 2. SELECT ROUND(15.193 , -1); - This will convert the number to nearest ten's .

Output

ROUND(15.193 , -1)
20

4. SIGN( ) – This function returns sign of a given number.

If number is negative, the function returns -1.

If number is positive, the function returns 1.

If number is zero, the function returns 0.

e.g. SELECT SIGN(-15) ;

Output

SIGN(-15)
-1

e.g.2 SELECT SIGN(20) ;

Output

SIGN(20)
1

5. SQRT( ) – This function returns the square root of a given number. E.g. SELECT SQRT(25) ;

Output

SQRT(25)
5

6. TRUNCATE( ) – This function returns a number with some digits truncated. E.g. SELECT TRUNCATE(15.79 , 1) ;

Output

TRUNCATE(15.79 , 1)
15.7

E.g. 2. SELECT TRUNCATE(15.79 , -1); - This command truncate value 15.79 to nearest ten's place.

Output

TRUNCATE(15.79 , -1)
10

DATE AND TIME FUNCTIONS

Date functions operate on values of the DATE datatype.

1. CURDATE( ) / CURRENT_DATE( ) – This function returns the current date. E.g.

```
SELECT CURDATE( ) ;
```

Output

CURDATE( )
2016-12-13

2. DATE( ) – This function extracts the date part from a date. E.g.

```
SELECT DATE( '2016-02-09') ;
```

Output

DATE('2016-02-09')
09

3. MONTH( ) – This function returns the month from the date passed. E.g.

```
SELECT MONTH( '2016-02-09') ;
```

Output

MONTH('2016-02-09')
02

4. YEAR( ) – This function returns the year part of a date. E.g.

```
SELECT YEAR( '2016-02-09') ;
```

Output

YEAR('2016-02-09')
2016

5. DAYNAME( ) – This function returns the name of weekday. E.g.

```
SELECT DAYNAME( '2016-02-09') ;
```

Output

DAYNAME('2016-12-14')
Wednesday

6. DAYOFMONTH( ) – This function returns the day of month. Returns value in range of 1 to 31.E.g.

```
SELECT DAYOFMONTH( '2016-12-14') ;
```

Output

DAYOFMONTH('2016-12-14')
14

7. DAYOFWEEK( ) – This function returns the day of week. Return the weekday index for date. (1=Sunday, 2=Monday,……., 7=Saturday)

```
SELECT DAYOFWEEK( '2016-12-14') ;
```

Output

DAYOFWEEK('2016-12-14')
4

8. DAYOFYEAR( ) – This function returns the day of the year. Returns the value between 1 and 366. E.g.

```
SELECT DAYOFYEAR('2016-02-04) ;
```

Output

DAYOFYEAR('2016-02-04')
35

9. NOW( ) – This function returns the current date and time.

It returns a constant time that indicates the time at which the statement began to execute.

e.g. SELECT NOW( );

10. SYSDATE( ) – It also returns the current date but it return the time at which SYSDATE( ) executes. It differs from the behavior for NOW( ), which returns a constant time that indicates the time at which the statement began to execute.

e.g. SELECT SYSDATE( ) ;

DATABASE TRANSACTIONS

TRANSACTION

A Transaction is a logical unit of work that must succeed or fail in its entirety. This statement means that a transaction may involve many sub steps, which should either all be carried out successfully or all be ignored if some failure occurs. A Transaction is an atomic operation which may not be divided into smaller operations.

Example of a Transaction

Begin transaction
- Get balance from account X
- Calculate new balance as X – 1000
- Store new balance into database file
- Get balance from account Y
- Calculate new balance as Y + 1000
- Store new balance into database file

End transaction

TRANSACTION PROPERTIES (ACID PROPERTIES)

1. **A**TOMICITY(All or None Concept) – This property ensures that either all operations of the transaction are carried out or none are.

2. **C**ONSISTENCY – This property implies that if the database was in a consistent state before the start of transaction execution, then upon termination of transaction, the database will also be in a consistent state.

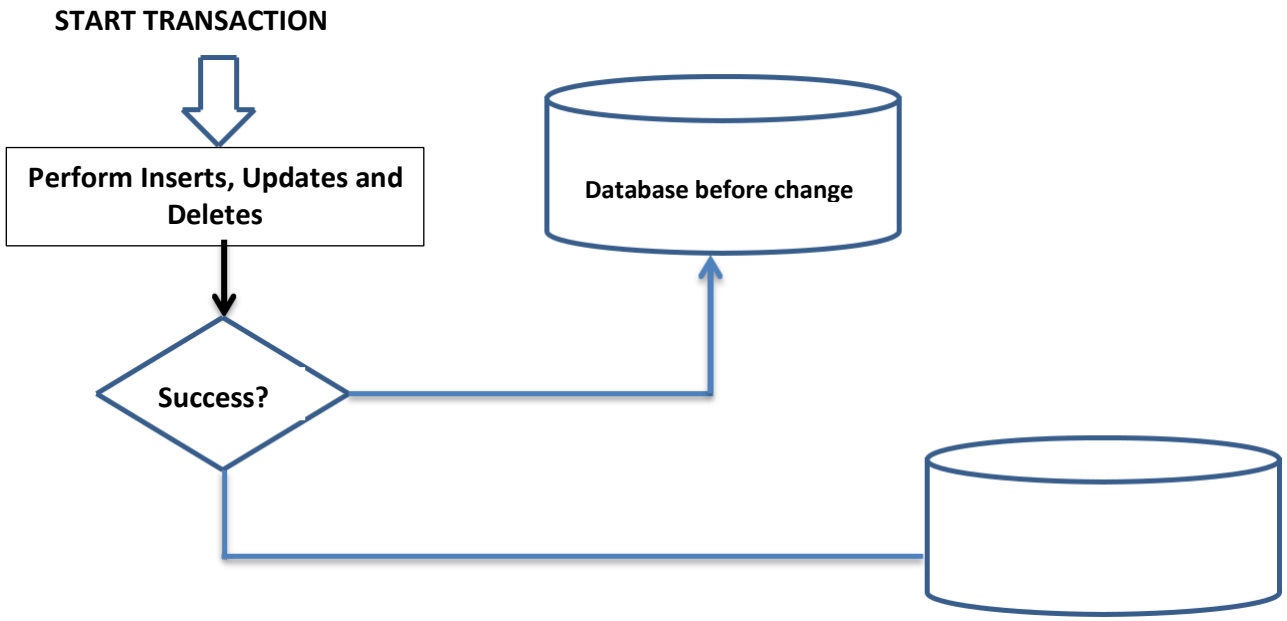
3. **I**SOLATION – This property implies that each transaction is unaware of other transactions executing concurrently in the system.

4. **D**URABILITY – This property of a transaction ensures that after the successful completion of a transaction, the changes made by it to the database persist, even if there are system failures.

TRANSACTION CONTROL COMMANDS(TCL)

The TCL of MySQL consists of following commands: BEGIN or START TRANSACTION – marks the beginning of a transaction.

1. COMMIT – Ends the current transaction by saving database changes and starts a new transaction.
2. ROLLBACK – Ends the current transaction by discarding database changes and starts a new transaction.
3. SAVEPOINT – Define breakpoints for the transaction to allow partial rollbacks.
4. SET AUTOCOMMIT – Enables or disables the default auto commit mode.



SET AUTOCOMMIT

By default, MySQL has autocommit ON, which means if you do not start a transaction explicitly through a BEGIN or STATE TRANSACTION command, then every statement is considered one transaction and is committed there and then.

You can check the current setting by executing the following statement :

mysql > select @@autocommit ;

@@autocommit
1

1 means, autocommit is enabled

To disable autocommit , command is:

SET autocommit = 0 ;

To enable autocommit, command is :

SET autocommit = 1 ;

Q1: Given a table t3(code, grade , value).

1	G	300
2	K	600
3	B	200

Considering table t3, for the following series of statements, determine which changes will become permanent, and what will be the output produced by last SELECT statement i.e., statement 7th.

1. SELECT * FROM T3 ;
2. INSERT INTO t3 VALUES(4, 'A' , 100) ;
4. ROLLBACK WORK ;DELETE FROM t3 WHERE CODE = 2;
5. DELETE FROM t3 WHERE CODE = 4;
6. ROLLBACK WORK;
7. SELECT * FROM t3 ;

Q2. Give one difference between ROLLBACK and COMMIT command used in MySQL

Q3. Given below is the 'Emp' table :

ENO	NAME
1	Anita Khanna
2	Bishmeet Singh

SET AUTOCOMMIT = 0 ;
INSERT INTO Emp VALUES(5 , 'Fazaria');
COMMIT ;
UPDATE Emp SET NAME = 'Farzziya' WHERE ENO = 5;
SAVEPOINTA ;
INSERT INTO Emp VALUES(6 , 'Richards');
SAVEPOINTB;
INSERT INTO Emp VALUES(7, 'Rajyalakshmi');
SAVEPOINTC ;
ROLLBACK TO B ;
What will be the output of the following SQL query now ? SELECT * FROM Emp ;

Q4. If you have not executed the COMMIT command , executing which command will reverse all updates made during the current work session in MySQL ?

Q5. What effect does SET AUTOCOMMIT have in transactions ?

Q6. What is a Transaction ? Which command is used to make changes done by a Transaction permanent on a database?

JOINS

- A join is a query that combines rows from two or more tables. In a join- query, more than one table are listed in FROM clause.

Table : empl

EMPNO	ENAME	JOB	SAL	DEPTNO
8369	SMITH	CLERK	2985	10
8499	ANYA	SALESMAN	9870	20
8566	AMIR	SALESMAN	8760	30
8698	BINA	MANAGER	5643	20

Table : dept

DEPTNO	DNAME	LOC
10	ACCOUNTING	NEW DELHI
20	RESEARCH	CHENNAI
30	SALES	KOLKATA
40	OPERATIONS	MUMBAI

CARTESIAN PRODUCT/UNRESTRICTED JOIN/CROSS JOIN

- Consider the following query :

  SELECT *
  FROM EMPL, DEPT ;

empno	ename	job	sal	deptno	deptno	dname	loc
8369	SMITH	CLERK	2985	10	10	ACCOUNTING	NEW DELHI
8499	ANYA	SALESMAN	9870	20	10	ACCOUNTING	NEW DELHI
8566	AMIR	SALESMAN	8760	30	10	ACCOUNTING	NEW DELHI
8698	BINA	MANAGER	5643	20	10	ACCOUNTING	NEW DELHI
8369	SMITH	CLERK	2985	10	20	RESEARCH	CHENNAI
8499	ANYA	SALESMAN	9870	20	20	RESEARCH	CHENNAI
8566	AMIR	SALESMAN	8760	30	20	RESEARCH	CHENNAI
8698	BINA	MANAGER	5643	20	20	RESEARCH	CHENNAI
8369	SMITH	CLERK	2985	10	30	SALES	KOLKATA
8499	ANYA	SALESMAN	9870	20	30	SALES	KOLKATA
8566	AMIR	SALESMAN	8760	30	30	SALES	KOLKATA
8698	BINA	MANAGER	5643	20	30	SALES	KOLKATA
8369	SMITH	CLERK	2985	10	40	OPERATIONS	MUMBAI
8499	ANYA	SALESMAN	9870	20	40	OPERATIONS	MUMBAI
8566	AMIR	SALESMAN	8760	30	40	OPERATIONS	MUMBAI
8698	BINA	MANAGER	5643	20	40	OPERATIONS	MUMBAI

This query will give you the Cartesian product i.e. all possible concatenations are formed of all rows of both the tables EMPL and DEPT. Such an operation is also known as **Unrestricted Join.** It returns n1 x n2 rows where n1 is number of rows in first table and n2 is number of rows in second table.

EQUI-JOIN

- The join in which columns are compared for equality, is called Equi - Join. In equi-join, all the columns from joining table appear in the output even if they are identical.

  e.g. SELECT * FROM empl, dept
  WHERE empl.deptno = dept.deptno ;

deptno column is appearing twice in output.

```
mysql> SELECT * FROM EMPL, DEPT WHERE EMPL.DEPTNO=DEPT.DEPTNO;
+-------+-------+----------+------+--------+--------+------------+-----------+
| empno | ename | job      | sal  | deptno | deptno | dname      | loc       |
+-------+-------+----------+------+--------+--------+------------+-----------+
|  8369 | SMITH | CLERK    | 2985 |     10 |     10 | ACCOUNTING | NEW DELHI |
|  8499 | ANYA  | SALESMAN | 9870 |     20 |     20 | RESEARCH   | CHENNAI   |
|  8698 | BINA  | MANAGER  | 5643 |     20 |     20 | RESEARCH   | CHENNAI   |
|  8566 | AMIR  | SALESMAN | 8760 |     30 |     30 | SALES      | KOLKATA   |
+-------+-------+----------+------+--------+--------+------------+-----------+
```

Q

Q: with reference to empl and dept table, find the location of employee SMITH.

ename column is present in empl and loc column is present in dept. In order to obtain the result, we have to join two tables.

SELECT ENAME, LOC
FROM EMPL, DEPT
WHERE EMPL.DEPTNO = DEPT.DEPTNO AND ENAME='SMITH';

```
+-------+-----------+
| ENAME | LOC       |
+-------+-----------+
| SMITH | NEW DELHI |
+-------+-----------+
```

Q: Display details like department number, department name, employee number, employee name, job and salary. And order the rows by employee number.

SELECT EMPL.deptno, dname,empno,ename,job,sal
FROM EMPL,DEPT
WHERE EMPL.DEPTNO=DEPT.DEPTNO
ORDER BY EMPL.DEPTNO;

```
+--------+------------+-------+-------+----------+------+
| deptno | dname      | empno | ename | job      | sal  |
+--------+------------+-------+-------+----------+------+
|     10 | ACCOUNTING |  8369 | SMITH | CLERK    | 2985 |
|     20 | RESEARCH   |  8698 | BINA  | MANAGER  | 5643 |
|     20 | RESEARCH   |  8499 | ANYA  | SALESMAN | 9870 |
|     30 | SALES      |  8566 | AMIR  | SALESMAN | 8760 |
+--------+------------+-------+-------+----------+------+
```

QUALIFIED NAMES

Did you notice that in all the WHERE conditions of join queries given so far, the field(column) names are given as: **<tablename>.<columnname>**

This type of field names are called qualified field names. Qualified field names are very useful in identifying a field if the two joining tables have fields with same time. For example, if we say deptno field from joining tables empl and dept, you'll definitely ask- **deptno** field of which table ? To avoid such an ambiguity, the qualified field names are used.

TABLE ALIAS

- A table alias is a temporary label given along with table name in FROM clause.

e.g.

```
SELECT E.DEPTNO, DNAME,EMPNO,ENAME,JOB,SAL
FROM EMPL E, DEPT D
WHERE E.DEPTNO = DEPT.DEPTNO
ORDER BY E.DEPTNO;
```

In above command table alias for EMPL table is E and for DEPT table , alias is D.

Q: Display details like department number, department name, employee number, employee name, job and salary. And order the rows by employee number with department number. These details should be only for employees earning atleast Rs. 6000 and of SALES department.

```
SELECT E.DEPTNO, DNAME,EMPNO, ENAME, JOB, SAL
FROM EMPL E, DEPT D
WHERE E.DEPTNO = D.DEPTNO
AND DNAME='SALES'
AND SAL>=6000
ORDER BY E.DEPTNO;
```

DEPTNO	DNAME	EMPNO	ENAME	JOB	SAL
30	SALES	8566	AMIR	SALESMAN	8760

NATURAL JOIN

By default, the results of an equijoin contain two identical columns. One of the two identical columns can be eliminated by restating the query. This result is called a Natural join.

e.g.
```
SELECT empl.*, dname, loc
FROM empl,dept
WHERE empl.deptno = dept.deptno ;
```

empno	ename	job	sal	deptno	dname	loc
8369	SMITH	CLERK	2985	10	ACCOUNTING	NEW DELHI
8499	ANYA	SALESMAN	9870	20	RESEARCH	CHENNAI
8698	BINA	MANAGER	5643	20	RESEARCH	CHENNAI
8566	AMIR	SALESMAN	8760	30	SALES	KOLKATA

empl.* means select all columns from empl table. This thing can be used with any table.

The join in which only one of the identical columns(coming from joined tables) exists, is called **Natural Join.**

LEFT, RIGHT JOINS

When you join tables based on some condition, you may find that only some, not all rows from either table match with rows of other table. When you display an equi join or natural join, it shows only the matched rows. What if you want to know which all rows from a table did not match with other. In such a case, MySQL left or right JOIN can be very helpful.

LEFT JOIN

- You can use LEFT JOIN clause in SELECT to produce left join
  i.e. SELECT <select-list>
  FROM <table1> LEFT JOIN <table2> ON <joining-condition>;
- When using LEFT JOIN all rows from the first table will be returned whether there are matches in the second table or not. For unmatched rows of first table, NULL is shown in columns of second table

S1

Roll_no	Name
1	A
2	B
3	C
4	D
5	E
6	F

S2

Roll_no	Class
2	III
4	IX
1	IV
3	V
7	I
8	II

```
SELECT S1.ROLL_NO, NAME,CLASS
FROM S1 LEFT JOIN S2 ON S1.ROLL_NO=S2.ROLL_NO;
```

```
+---------+------+-------+
| ROLL_NO | NAME | CLASS |
+---------+------+-------+
|       1 | A    | IV    |
|       2 | B    | III   |
|       3 | C    | V     |
|       4 | D    | IX    |
|       5 | E    | NULL  |
|       6 | F    | NULL  |
+---------+------+-------+
6 rows in set (0.00 sec)
```

RIGHT JOIN

- It works just like LEFT JOIN but with table order reversed. All rows from the second table are going to be returned whether or not there are matches in the first table.
- You can use RIGHT JOIN in SELECT to produce right join
  i.e. SELECT <select-list>
  FROM <table1> RIGHT JOIN <table2> ON <joining-condition>;

e.g

```
SELECT S1.ROLL_NO, NAME,CLASS
FROM S1 RIGHT JOIN S2 ON S1.ROLL_NO=S2.ROLL_NO;
```

```
+---------+------+-------+
| ROLL_NO | NAME | CLASS |
+---------+------+-------+
|       2 | B    | III   |
|       4 | D    | IX    |
|       1 | A    | IV    |
|       3 | C    | V     |
|    NULL | NULL | I     |
|    NULL | NULL | II    |
+---------+------+-------+
```

Q: In a database there are two tables:
Table: ITEM

Item_Code	Item_Name	Price
111	Refrigerator	90000
222	Television	75000
333	Computer	42000
444	Washing Machine	27000

Table:BRAND

Item_Code	Brand_Name
111	LG
222	Sony
333	HCL
444	IFB

Write MySql queries for the following:

(i) To display Item_Code, Item_Name and corresponding Brand_Name of those Items, whose price is between 20000 and 40000 (both values inclusive).

(ii) To display Item_Code, Price and Brand_Name of the item, which has Item_Name as "Computer".

(iii) To increase the price of all items by 10%.

Q: A table "Transport" in a database has degree 3 and cardinality 8. What is the number of rows and columns in it ?

Q: Table Employee has 4 records and Table Dept has 3 records in it. Mr. Jain wants to display all information stored in both of these related tables. He forgot to specify equi - join condition in the query. How many rows will get displayed on execution of this query ?

Q: In a database there are two tables "ITEM" and "CUSTOMER" as shown below:

Table: ITEM

ID	ItemName	Company	Price
1001	Moisturiser	XYZ	40
1002	Sanitizer	LAC	35
1003	Bath Soap	COP	25
1004	Shampoo	TAP	95
1005	Lens Solution	COP	350

Table: CUSTOMER

C_ID	CustomerName	City	ID
01	Samridh Ltd	New Delhi	1002
05	Big Line Inc	Mumbai	1005
12	97.8	New Delhi	1001
15	Tom N Jerry	Bangalore	1003

Write the commands in SQL queries for the following:

(i) To display the details of Item, whose price is in the range of 40 and 95 (Both values included).

(ii) To display the customername, city from table customer and ItemName and Price from table Item with their corresponding matching ID.

(iii) To increase the price of all the Products by 50.

Q: A table FLIGHT has 4 rows and 2 columns and another table AIR hostess has 3 rows and 4 columns. How many rows and columns will be there if we obtain the Cartesian product of these two tables ?

Q: Given below is the Table Patient.

Table : Patient

Name	P_No	Date_Admn	Doc_No
Mrs. Vimla Jain	P0001	2011-10-11	D201
Miss Ishita Kohli	P0012	2011-10-11	D506
Mr. Vijay Verma	P1002	2011-10-17	D201
Mr. Vijay Verma	P1567	2011-11-22	D233

(i) Identify Primary Key in the table given above.

(ii) Write MySql query to add a column Department with data type varchar and size 30 in the table patient.

Q: Write MySql command to create the Table Product including its Constraints.

Table: PRODUCT

Name of Column	Type	Size	Constraint
P_Id	Decimal	4	Primary Key
P_Name	Varchar	20	
P_Company	Varchar	20	
Price	Decimal	8	Not Null

Q: Write a MySQL command for creating a table 'PAYMENT' whose structure is given below:

Table : PAYMENT

Field Name	Datatype	Size	Constraint
Loan_number	Integer	4	Primary Key
Payment_number	Varchar	3	
Payment_date	Date		Not Null
Payment_amount	Integer	8	

Q:Write SQL command to create the Table Vehicle with given constraint.

Table : CHALLAN

COLUMN_NAME	DATATYPE(SIZE)	CONSTRAINT
Challan_No	Decimal(10)	Primary Key
Ch_Date	Date	
RegNo	Char(10)	
Offence	Decimal(3)	

Q: Consider the tables HANDSETS and CUSTOMER given below:

Table : Handsets

SetCode	SetName	TouchScreen	PhoneCost
N1	Nokia 2G	N	5000
N2	Nokia 3G	Y	8000
B1	BlackBerry	N	14000

Table: Customer

CustNo	SetNo	CustAddress
1	N2	Delhi
2	B1	Mumbai
3	N2	Mumbai
4	N1	Kolkata
5	B1	Delhi

With reference to these tables, Write commands in SQL for (i) and (ii) and output for (iii) below:

(i) Display the CustNo, CustAddress and Corresponding SetName for each customer.

(ii) Display the Cusomer Details for each customer who uses a Nokia handset.

(iii) Select Setno, SetName From Handsets, Customer
Where setno =setcode AND custAddress = 'Delhi';

Q: In a database there are two tables Company and Model as shown below:

Table : Company

CompID	CompName	CompHO	ContPerson
1	Titan	Okhla	CB. Ajit
2	Maxima	Shahdara	V.P.Kohli
3	Ajanta	Najafgarh	R.Mehta

Table : Model

ModelID	CompID	ModelCost
T020	1	2000
M032	4	2500
M059	2	7000

(i) Identify the foreign key column in the table model.

(ii) Check every value in CompID column of both the tables.Do you find any discrepancy ?

(iii) How many rows and columns will be there in the Cartesian product of these two tables ?

(iv) Write SQL command to change Model cost to 3500 for ModelID T020 in Model Table.